Evaluating the Efficacy of Machine Learning Algorithms in Detecting DDoS Attack Using CIC-DDoS2019 Dataset.

Suchitra Kairi (212056) Tahrima Arif Mim (220205) Jessica Sen (220241)

Group Number: 2

***Abstract*—Distributed Denial-of-Service (DDoS) attacks remain among the most disruptive threats to modern network infrastructure, and traditional signature-based intrusion detection systems increasingly struggle against fast-evolving, multi-vector attack campaigns. This study presents an empirical evaluation of four machine learning classifiers — Decision Tree, Random Forest, XGBoost, and LightGBM — for DDoS detection on the CIC-DDoS2019 benchmark dataset, addressing three gaps in the literature: the absence of systematic multi-algorithm comparisons, limited application of structured feature selection, and the near-total lack of cross-attack-type generalisation testing. A complete, reproducible pipeline was implemented covering data cleaning, multi-class baseline classification (RQ1), a three-stage filter-wrapper-embedded feature selection process (RQ2), and cross-attack-type generalisation with fine-tuning on a held-out DNS reflection attack (RQ3). Decision Tree, Random Forest, and XGBoost achieved comparable multi-class accuracy of approximately 0.74, while LightGBM underperformed at 0.18 under default hyperparameters, a limitation traced to its leaf-wise tree growth strategy under severe class imbalance. Feature selection reduced the feature space by 81% (77 to 15 features) with negligible accuracy loss and up to 64% faster training. A binary classifier generalised to the unseen DNS attack with 95.5% zero-shot accuracy, improving to 98.4% after fine-tuning with only 20% of target-domain data. These findings demonstrate that lightweight, interpretable tree-based classifiers can achieve competitive detection performance with substantially reduced computational overhead, and that limited target-domain fine-tuning offers a practical route to rapid defence against novel attack variants.**

Keywords: DDoS detection, machine learning, CIC-DDoS2019, Decision Tree, Random Forest, XGBoost, LightGBM, feature selection, network intrusion detection, cross-attack generalisation, fine-tuning.

I. INTRODUCTION

Distributed Denial-of-Service (DDoS) attacks represent one of the most persistent and operationally disruptive threats in modern network security. By flooding target systems with artificially high volumes of malicious traffic, adversaries render critical services — including financial platforms, healthcare portals, and government networks — temporarily or permanently unavailable, causing significant financial damage and reputational harm [10], [12]. As attack methodologies evolve in complexity and scale, traditional signature-based and rule-based intrusion detection systems have proven increasingly inadequate, since they rely on static pattern matching that cannot adapt to novel or multi-vector attacks. Machine learning offers adaptive, data-driven classification that can generalise beyond pre-defined attack signatures; among these approaches, tree-based ensembles — Decision Tree, Random Forest, XGBoost, and LightGBM — have demonstrated strong performance on benchmark network traffic datasets [1], [2].

Critical gaps nonetheless remain. Most existing studies evaluate only a single algorithm without systematic comparison, and comparative studies that do exist rarely report computational overhead (training time, inference time, memory usage) alongside accuracy, limiting their usefulness for real-time deployment decisions [1], [3], [9]. Few studies investigate the impact of structured feature selection on detection quality and computational cost: feature selection has been shown to reduce dimensionality without substantial accuracy loss [6], [8], yet no study combines a structured, multi-stage feature selection pipeline with a systematic multi-algorithm comparison. Most critically, almost none assess how well classifiers trained on one DDoS sub-type generalise to unseen attack categories [7], [11]; existing generalisation studies report that classifiers trained on one attack distribution collapse to near-random performance when evaluated on another [7], with no attempt made to recover performance through lightweight fine-tuning. This study addresses all three gaps through empirical experimentation on CIC-DDoS2019, a publicly available, large-scale, labelled benchmark containing diverse DDoS attack sub-types.

This study acquires and cleans the CIC-DDoS2019 dataset, implements and compares four classifiers on accuracy, false-positive rate, and computational cost; applies a three-stage (filter, wrapper, embedded) feature selection pipeline and evaluates its efficiency impact; and evaluates the generalisation capability of the best classifier on an unseen attack type before and after fine-tuning with a small proportion of target-specific data.

This study is guided by the following three research questions:

- **RQ1:** How do Decision Tree, Random Forest, XGBoost, LightGBM algorithms compare the accuracy and false-positive rates when classifying DDoS attack traffic in the CIC-DDoS2019 dataset, and how do these algorithms compare in terms of training time and computational overhead?
- **RQ2:** How does a three-stage feature selection approach affect the detection accuracy and computational efficiency of different machine learning classifiers compared with using the complete feature set?
- **RQ3:** How does the performance of best classifiers vary when detecting unseen DDoS attack types, and

to what extent can fine tuning this classifier with a little amount of target-specific malicious data improve detection performance?

The major contributions of this paper are: a systematic four-algorithm comparison jointly reporting accuracy, false-positive rate, and computational overhead; a structured three-stage feature selection pipeline that extends the two-stage hybrid approach of Sakr et al. [6] to an 81% feature reduction with a benchmarked efficiency gain; a cross-attack-type generalisation experiment with a demonstrated lightweight fine-tuning recovery mechanism, directly addressing the collapse documented by Cantone et al. [7]; and a transparent, reproducible pipeline with honest disclosure of model limitations.

II. RELATED WORK

This section synthesises existing research on machine-learning-based DDoS detection, organised thematically around three areas directly relevant to this study's research questions: algorithm comparison, feature selection, and cross-dataset generalisation.

A. ML Algorithm Comparison for DDoS Detection:

Saluja et al. [1] found tree-based models outperform probabilistic and distance-based methods on CIC-DDoS2019 but reported no efficiency metrics or feature selection, leaving open questions about real-time deployability. Al-Eryani et al. [2] reported near-perfect accuracy (99.98% XGBoost, 99.99% Gradient Boosting) on a pre-sampled 360,000-record subset, a sampling choice that can inflate accuracy by narrowing the class imbalance present in the full dataset. Kapil et al. [3] confirmed Random Forest's strength across UDP, SYN, NetBIOS, and LDAP sub-types on the full feature set, but applied no feature selection and made no attempt at cross-attack-type generalisation. Meng et al. [4] found XGBoost outperforms LightGBM in precision and recall on imbalanced network traffic, while LightGBM trains faster and uses less memory — a trade-off directly corroborated by RQ1 of this study. Devi et al. [9] showed hybrid-estimator accuracy gains on CIC-DDoS2019 without reporting inference time or memory, and Abualhassan et al. [5] confirmed classical ML matches deep learning accuracy at substantially lower resource cost on IoT DDoS traffic, justifying the tree-based focus adopted here.

B. Feature Selection Approaches in DDoS:

Sakr et al. [6] reduced CIC-DDoS2019 features from 79 to 35 via a hybrid filter-wrapper approach while maintaining accuracy, acknowledging that wrapper methods incur significant computational cost — the efficiency trade-off directly investigated in RQ2. Prasad and Chandra [8] applied embedded LightGBM feature importance with SMOTE balancing for IoT DDoS detection but did not compare the embedded method against filter or wrapper alternatives in a controlled setup, a comparison the present three-stage pipeline provides directly. Deressu et al.'s [10] 47-study survey found ML-based methods now dominate DDoS detection research (31%, followed by hybrid and deep learning at 28.6% each) and identified that most benchmarks focus on single attack types and lack multi-sub-type evaluation, a gap this study's multi-class experimental design addresses.

C. Generalisation, Transfer Learning, and Cross-Attack Robustness:

Cantone et al. [7] found cross-dataset accuracy collapses to near-random levels for most attack categories despite near-perfect within-dataset performance, with no fine-tuning or domain adaptation attempted to recover performance — the central theoretical motivation for RQ3. Anley et al. [11] demonstrated that transfer learning between CIC-DDoS2019 and KDDCup'99 using CNN architectures (VGG16, VGG19, ResNet50) with adaptive hyperparameter optimisation consistently improved performance, but relies on deep learning rather than classical ML, making results difficult to compare with tree-based classifiers. Bouyeddou et al. [12] surveyed ML-based DDoS detection in Software-Defined Networking, identifying accuracy, scalability, and generalisation as the field's three most persistent open challenges — findings that validate all three research questions of this study.

TABLE I: Summary of the Research Gap Across directly Comparable Studies.

Study	Core Approach	Key Limitation / Gap
Saluja et al. [1]	RF/DT/NB/KNN comparison	No feature selection; no efficiency metrics; no generalisation test.
Al-Eryani et al. [2]	XGBoost, Gradient Boosting	Pre-sampled 360k-record subset; no generalisation test.
Sakr et al. [6]	Filter + wrapper feature selection	No cross-attack-type generalisation experiment.
Cantone et al. [7]	Cross-dataset generalisation study	No fine-tuning or recovery mechanism attempted.
Anley et al. [11]	CNN-based transfer learning	Deep learning only; not applied to classical ML.
This Study	**4-algorithm comparison + 3-stage feature selection + generalisation with fine-tuning**	**—**

No single prior study combines a systematic multi-algorithm comparison, a structured multi-stage feature selection pipeline benchmarked for efficiency, and a cross-attack-type generalisation evaluation with an explicit fine-tuning recovery mechanism — the combination this study provides.

III. METHODOLOGY AND IMPLEMENTATION

A. System Architecture:

Fig. 1 shows the complete pipeline, from dataset ingestion through the three RQ-specific branches to shared evaluation and reporting.

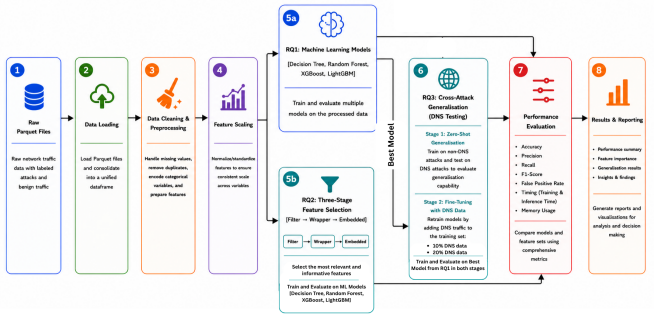

Fig. 1: Proposed system architecture.

B. Experimental Setup:

All experiments were executed in Google Colaboratory, with the CIC-DDoS2019 Parquet files stored on and accessed

via a mounted Google Drive directory, on Colab's standard CPU runtime with no dedicated GPU hardware, a specification disclosed here as relevant to the reported training and inference times. The implementation was written in Python 3, using pandas and NumPy for data handling; scikit-learn for Decision Tree, Random Forest, feature scaling, label encoding, mutual information scoring, recursive feature elimination, and evaluation metrics; XGBoost and LightGBM for the corresponding gradient-boosted classifiers; joblib for model serialisation; psutil for process memory measurement; and matplotlib for all figures. The dataset used was CIC-DDoS2019, comprising CICFlowMeter-derived network flow records with 77 features per flow. Six known attack sub-types were used for RQ1 and RQ2 — LDAP, MSSQL, NetBIOS, Syn, UDP, and UDP-Lag — while a seventh, DNS reflection, was deliberately withheld from all RQ1/RQ2 training and testing to serve as a genuinely unseen attack type for the RQ3 generalisation experiment.

C. Research Procedure:

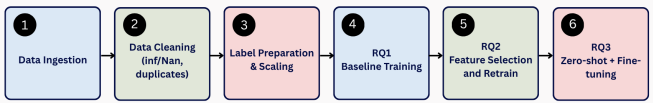

Fig. 2: Sequential Research Procedure.

Data is ingested per attack type, cleaned (infinite/missing values and duplicates removed), merged with shared label encoding and feature scaling, then branched into RQ1 baseline training, RQ2 feature-selected retraining, and RQ3 zero-shot plus fine-tuned generalisation testing on the held-out DNS partition.

D. Configuration and Hyperparameters:

Table II documents and justifies the critical parameters and settings used throughout the study.

TABLE II: Configuration Parameters and Justifications.

Parameter	Value	Justification
Random state (all models/splits)	42	Reproducibility across models/splits.
RF / LightGBM / XGBoost	200 estimators	Fair like-for-like comparison.
XGBoost – objective, tree_method	`multi:softprob`, `hist`	Supports multi-class classification and improves training speed.
Filter→Wrapper→Embedded	77→40→25→15	Hybrid pipeline per Sakr et al. [6].
Fine-tuning samples	10% / 20% stratified	Tests value of small target data (RQ3).
Metric averaging	Weighted	Accounts for class imbalance across seven attack classes.

E. Evaluation Metrics:

Accuracy, weighted precision, weighted recall, and weighted F1-score were computed to assess overall and class-imbalance-aware classification performance, with weighted averaging chosen over macro averaging so that headline metrics reflect the classes that dominate evaluation given the imbalance in Table III. False Positive Rate was derived from the confusion matrix as the mean, across all classes, of FP/(FP+TN), directly answering the false-positive component of RQ1. Training time and inference time (seconds) were measured via wall-clock timing to answer the computational-overhead components of RQ1 and RQ2, and memory usage (megabytes) was measured as the change in process resident set size before and after training, via the psutil library. For RQ3, the same core metrics were recomputed for the binary Benign-versus-Attack task, before and after each fine-tuning stage.

F. Implementation Strategy:

The implementation is a single, linearly executed notebook divided into clearly labelled sections — Setup, Data Cleaning, RQ1, RQ2, and RQ3 — so that every reported result can be traced to the exact cell that produced it. The merged multi-class training and testing frames are constructed once during RQ1 and reused unchanged as the starting point for RQ2, ensuring the two experiments are evaluated on identical rows; the StandardScaler and LabelEncoder fitted during RQ1 are likewise reused, not refitted, during RQ2, so both experiments operate on an identical feature representation. RQ3 is intentionally decoupled from this scaler, fitting its own on the binary training set, since it operates on a disjoint label space and a held-out attack type.

G. Design Decisions:

Several design decisions are disclosed explicitly for transparency. The test partition's 51 WebDDoS rows, which have no matching training examples, were removed prior to evaluation, since no classifier can be fairly assessed on a label it never saw during training. The DNS partition was withheld from all RQ1/RQ2 training and testing from the outset specifically to serve as a genuinely unseen attack type for RQ3; only a DNS testing partition exists in the source data, which does not limit RQ3 since DNS was needed only as unseen evaluation data, never for training. Because DNS reflection traffic has no corresponding multi-class training label, RQ3 was framed as binary classification (Benign versus Attack) rather than multi-class sub-type identification, a deliberate scope simplification revisited in the Discussion and Limitations sections. Weighted rather than macro metric averaging was used given the severe class imbalance in both the training set (Syn: 48,840 rows versus UDP-lag: 55 rows) and, independently, the testing set (UDP-lag: 8,872 rows versus Syn: 533 rows), so headline metrics reflect the classes that dominate evaluation. Each RQ3 fine-tuning stage retrains the Decision Tree from scratch on the combined dataset rather than using incremental learning, since scikit-learn's implementation provides no partial-fit capability; this full-retrain approach remains practical, since only 10–20% of new target-domain data was required to substantially recover performance. Finally, LightGBM retained default hyperparameters throughout, matching XGBoost's estimator count for a fair like-for-like comparison, without additional class-imbalance-specific tuning; the resulting performance gap is reported honestly in Sections IV and V rather than adjusted after the fact.

IV. RESULTS

All tables and figures below are direct, unaltered outputs of the executed pipeline. No infinite, missing, or duplicate rows were found in any of the six training partitions. After

merging and removing WebDDoS, the final dataset comprised 120,065 training and 38,922 testing rows across 77 features and seven classes, with the imbalanced distribution shown in Table III.

TABLE III: Class Distribution (RQ1/RQ2).

Class	Train	Test
Benign	42,007	10,847
Syn	48,840	533
DrDoS_UDP	18,090	10,420
DrDoS_MSSQL	8,523	6,212
DrDoS_LDAP	1,906	1,440
DrDoS_NetBIOS	644	598
UDP-lag	55	8,872

A. RQ1 — Baseline Multi-Class Classification:

TABLE IV: RQ1 Results (Full 77-Feature Set).

Model	Acc.	F1	FPR	Train(s)	Mem(MB)
Decision Tree	0.738	0.690	0.042	3.04	0.97
Random Forest	0.741	0.647	0.051	49.56	64.22
XGBoost	0.748	0.671	0.046	61.08	20.69
LightGBM	0.180	0.177	0.131	47.91	69.09

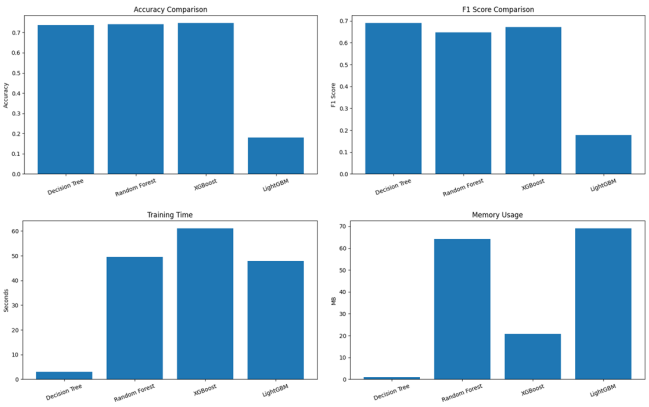

Fig. 3: RQ1 (a) accuracy, (b) F1, (c) training time, (d) memory across models.

Decision Tree, Random Forest, and XGBoost converge to a similar accuracy band (0.738–0.748); LightGBM is a clear outlier at 0.180, close to random-guess performance on a seven-class problem, examined mechanistically in Section V-A. Decision Tree trains and infers dramatically faster than the ensemble methods — 16 to 20× faster than Random Forest and XGBoost — for an accuracy cost of only 0.3–1.0 percentage points, and also achieves the lowest false-positive rate (0.042) of the four models, directly answering RQ1's overhead and false-positive questions.

B. RQ2 — Three-Stage Feature Selection:

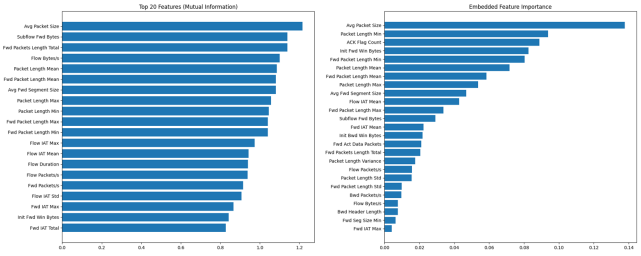

Fig. 4: (a) Filter stage — top mutual-information features; (b) Embedded stage — final 15 features.

TABLE V: RQ2 Results (15-Feature Set)

Model	Acc.	F1	FPR	Train(s)
Decision Tree	0.739	0.692	0.042	0.57
Random Forest	0.741	0.647	0.050	18.97
XGBoost	0.739	0.646	0.051	21.75
LightGBM	0.574	0.513	0.077	13.52

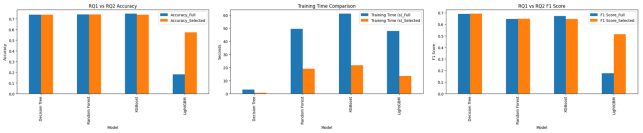

Fig. 5: RQ1 vs. RQ2: (a) accuracy, (b) training time, (c) F1 Score.

Accuracy is essentially unchanged for Decision Tree, Random Forest, and XGBoost despite using only 19% of the original features, while training time decreases sharply: Random Forest falls from 49.56s to 18.97s (-62%), and XGBoost from 61.08s to 21.75s (-64%). LightGBM improves substantially, from 0.180 to 0.574 accuracy, once the feature space is reduced, supporting the hypothesis that its RQ1 failure relates to split-search difficulty rather than fundamental unsuitability. Weighted F1 remains near 0.69 for Decision Tree despite 74% accuracy because Syn and UDP-lag exhibit near-zero recall or precision, a direct consequence of the class-count mismatch in Table III. The final 15 retained features are dominated by packet-size and window-size statistics rather than flow-timing features.

C. RQ3 — Cross-Attack-Type Generalisation:

TABLE VI: RQ3 Results (Binary, Held-Out DNS Traffic)

Stage	Acc.	Prec.	Recall	F1
Zero-shot	0.9554	0.9864	0.9313	0.9581
10% FT	0.9733	0.9956	0.9555	0.9751
20% FT	0.9838	0.9976	0.9728	0.9850

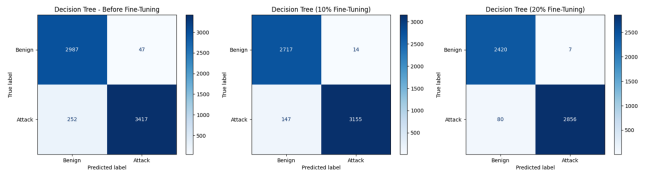

Fig. 6: Confusion matrices: (a) zero-shot, (b) 10%, (c) 20% fine-tuned.

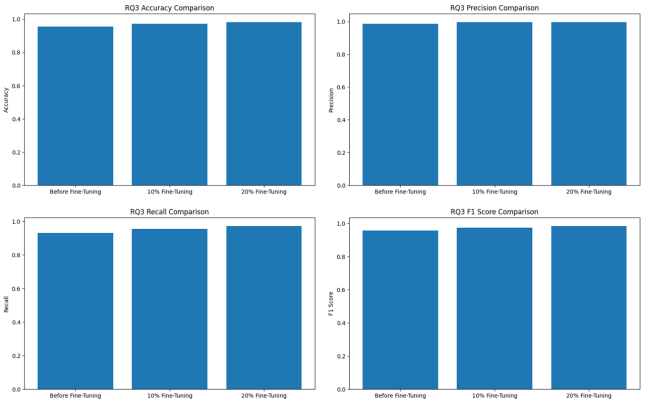

Fig. 7: RQ3 (a) accuracy, (b) precision, (c) recall, (d) F1 across the three stages.

A classifier trained only on known DDoS sub-types detects the unseen DNS attack with 95.5% accuracy and 93.1% recall in a zero-shot setting, well above the near-random cross-dataset collapse reported by Cantone et al. [7], though this reflects an easier binary task than full sub-type identification. Fine-tuning with a stratified 10% and then 20% sample of DNS traffic improves recall monotonically to 95.5% and 97.3% respectively, while precision remains at or above 98.6% throughout, indicating the recall improvement is not achieved at the cost of additional false alarms. These results are interpreted mechanistically, and compared against the literature, in Section V.

V. DISCUSSION

A. Interpreting RQ1:

The comparable accuracy achieved by Decision Tree, Random Forest, and XGBoost indicates that, for the volumetric flood attacks in CIC-DDoS2019, ensembling contributes only marginal discriminative benefit over a single well-fitted tree; such floods produce statistically distinctive packet-size and flow-volume patterns that a single tree already exploits effectively, leaving comparatively little residual signal for ensembling to capture. XGBoost's marginally higher accuracy likely reflects its gradient-boosted correction of residual errors across successive trees, at the cost of substantially higher training time. LightGBM's pronounced underperformance is attributable to its leaf-wise tree growth strategy, which greedily expands whichever leaf offers the largest loss reduction — a strategy that requires sufficient training samples per leaf to identify statistically valid splits. Several classes are severely underrepresented in training (UDP-lag: 55 samples; DrDoS_NetBIOS: 644 samples), a condition under which leaf-wise growth is known to fail to identify positive-gain splits, precisely the behaviour recorded in the training logs. This diagnosis is corroborated by RQ2: reducing the feature space from 77 to 15 dimensions raised LightGBM's accuracy from 0.180 to 0.574, since a lower-dimensional split search space made valid splits considerably easier to locate even with limited per-class samples.

B. Interpreting RQ2:

The negligible accuracy loss following an 81% feature reduction indicates substantial redundancy within the original 77-feature representation; many CICFlowMeter-derived features (e.g., Packet Length Minimum, Maximum, Mean, and Standard Deviation) are highly correlated descriptors of the same underlying packet-size distribution, so once a compact subset capturing that distribution is retained, the remaining correlated features add little discriminative value. This extends Sakr et al.'s [6] 56% reduction (79→35 features) to a substantially more aggressive 81% rate while still preserving detection accuracy on CIC-DDoS2019. The retained features are dominated by packet-size and TCP window-size statistics rather than flow-timing features, reflecting the operational signature of volumetric flood tools, which typically generate traffic with narrow, scripted packet-size distributions and non-standard window-size behaviour, since automated flooding software rarely implements the full congestion-control logic of a legitimate TCP stack. Flow-timing features contributed comparatively little discriminative value here because the represented attacks are defined primarily by abnormal packet volume and structure rather than abnormal inter-arrival timing — implying the retained feature set would likely perform poorly against low-and-slow application-layer attacks such as Slowloris, which deliberately mimic normal packet sizes while manipulating timing.

C. Interpreting RQ3:

The 95.5% zero-shot accuracy achieved on the withheld DNS attack is materially more meaningful than high accuracy on previously seen attack types, because it evaluates the classifier's capacity to generalise beyond its training distribution rather than merely recall patterns it has already observed — a capability trivially satisfied by any sufficiently expressive model and one that provides no evidence of practical deployability against novel threats. This result should be interpreted precisely: because the classifier is trained as a binary Benign-versus-Attack detector using only volumetric flood examples, its correct classification of DNS traffic as malicious most plausibly reflects recognition that the traffic deviates statistically from benign behaviour, rather than specific recognition of DNS reflection as a distinct attack sub-type. The monotonic improvement in recall across the zero-shot, 10%, and 20% fine-tuning stages (93.1%, 95.5%, and 97.3% respectively), achieved while precision remained at or above 98.6% throughout, indicates that fine-tuning specifically recovers previously missed attack instances without introducing additional false alarms — a materially more favourable outcome than the near-random cross-dataset collapse reported by Cantone et al. [7], who attempted no recovery mechanism.

D. Comparison with the Literature:

Al-Eryani et al. [2] reported near-perfect accuracy (99.98% XGBoost, 99.99% Gradient Boosting) on CIC-DDoS2019, substantially exceeding the 0.748 XGBoost accuracy obtained here. This discrepancy is attributable primarily to experimental design rather than a methodological shortcoming: Al-Eryani et al. [2] evaluated on a pre-sampled 360,000-record subset, a strategy that can inflate accuracy by reducing class-imbalance severity and narrowing the train-test distributional mismatch documented in Table III (e.g., Syn comprises 48,840 training rows but only 533 testing rows). The present study deliberately retained this imbalance to report performance under realistic, unmitigated conditions, a choice that produces lower headline accuracy but arguably higher external validity. Saluja et al. [1] similarly found tree-based dominance over probabilistic and distance-based methods, directionally consistent with this study, though neither computational efficiency nor false-positive rates were reported, limiting further quantitative comparison. Sakr et al.'s [6] 56% feature reduction (79→35 features) on a related IoT intrusion detection task achieved comparable accuracy preservation to that observed here, but at a substantially less aggressive reduction rate and without any accompanying training-time benchmark; the present study's 81% reduction, paired with the up-to-64% training-time savings reported in Section IV-B, quantifies an efficiency benefit that Sakr et al. [6] left unmeasured. With respect to generalisation, this study's 95.5% zero-shot accuracy and subsequent fine-tuning recovery stand in direct contrast to Cantone et al. [7], whose cross-dataset evaluation found accuracy collapsing to near-random with no recovery attempted — though this comparison should be qualified, since the present study evaluates generalisation to an unseen attack within the same dataset and feature schema, a less demanding task than Cantone et al.'s [7] cross-dataset evaluation across differently instrumented captures.

E. Operational Context for a SOC:

Interpreted for deployment in a Security Operations Centre, the false-positive rates achieved by the top classifiers (0.042–0.051, Table IV) indicate approximately 4–5% of benign flows would be misclassified as malicious; at production flow volumes, even this comparatively low rate would generate a substantial daily volume of false alerts, a well-documented contributor to analyst alert fatigue. More critically, the weighted F1-scores reported here (0.69 for Decision Tree) can obscure severe per-class failures: because weighted averaging proportionally discounts underrepresented classes, a classifier achieving near-zero recall on an entire attack category — as observed for Syn and UDP-lag, a direct consequence of the train-test mismatch in Table III — can nonetheless report a superficially acceptable aggregate score. An analyst relying on an aggregate dashboard metric would have no visibility into an entire undetected attack category, and would likely discover the gap only after operational impact occurred, underscoring that per-class recall, not aggregate weighted metrics alone, should be monitored in any operational deployment.

F. Practical Utility Despite Limitations:

Despite the limitations discussed below, the findings support a clear practical contribution. The demonstrated 81% feature reduction with negligible accuracy loss and up to 64% reduction in training time translates directly into lower computational cost, simpler feature-collection pipelines, and higher potential throughput for real-time detection, properties directly relevant to resource-constrained or high-volume deployment environments. Similarly, the demonstrated capacity to recover strong detection performance against a novel attack using only 10–20% of target-domain data offers a materially faster operational response pathway than full dataset collection and retraining, of particular value during the early window following the emergence of a new attack variant, when labelled data is by definition scarce. Taken together, lightweight, interpretable tree-based classifiers, deployed alongside per-class monitoring and periodic lightweight fine-tuning, represent a practically viable approach to DDoS detection.

VI. LIMITATION AND FUTURE WORK

Several limitations are disclosed explicitly, consistent with sound scientific practice. LightGBM's 18.0% accuracy under default hyperparameters (recovering to 57.4% after feature selection) indicates a clear need for further hyperparameter tuning — class weighting, min_child_samples, and num_leaves are identified as necessary future work rather than attempted retroactively within this study. The near-zero recall observed for Syn and UDP-lag stems from a pronounced mismatch between their training and testing row counts, inherited from the underlying CIC-DDoS2019 train/test day partitions; this is a property of the benchmark dataset itself rather than a modelling flaw, and constrains what any classifier trained under this split can achieve for those two classes. RQ3's binary framing tests general anomaly detection rather than DNS-specific sub-type identification, and because the retained feature set emphasises packet-size and window-size statistics, the evaluated classifiers would be expected to perform poorly against timing-based low-and-slow attacks such as Slowloris, which deliberately mimic normal packet sizes while manipulating connection timing. Fine-tuning was evaluated only for Decision Tree, the fastest of the four classifiers, to keep the experiment tractable within the project timeline; repeating it for Random Forest, XGBoost, and LightGBM is planned as an extension. Finally, all experiments ran on Google Colab's standard CPU runtime rather than dedicated GPU or production-grade hardware, using the six-attack-plus-DNS subset available in the project's dataset folder rather than the full CIC-DDoS2019 release, since several additional attack types (e.g., NTP, SNMP, SSDP) had missing or incomplete training/testing files. Future work should prioritise systematic hyperparameter tuning of LightGBM to close the observed performance gap; extending the fine-tuning experiment to Random Forest, XGBoost, and LightGBM; evaluating full multi-class attack-sub-type generalisation rather than binary anomaly detection; incorporating timing-sensitive features or a complementary detection mechanism to address the low-and-slow attack blind spot; and expanding the dataset

and evaluation environment to additional attack types and production-representative hardware as data and resources become available.

VII. CONCLUSION

This study set out to address three gaps in the machine-learning-based DDoS detection literature: the absence of systematic multi-algorithm comparisons that jointly report accuracy and computational overhead, the limited application of structured feature selection, and the near-total lack of cross-attack-type generalisation testing with a recovery mechanism. A complete, reproducible pipeline was implemented and executed on CIC-DDoS2019 to address these gaps directly. Decision Tree, Random Forest, and XGBoost achieved comparable multi-class accuracy of approximately 0.74, with Decision Tree offering the most favourable speed and false-positive-rate trade-off; LightGBM underperformed substantially under default hyperparameters, a limitation traced to its leaf-wise tree growth strategy under severe class imbalance. A three-stage filter-wrapper-embedded feature selection pipeline reduced the feature set from 77 to 15 features, an 81% reduction, while preserving detection accuracy and reducing training time by up to 64%. A binary classifier generalised to a previously unseen DNS reflection attack with 95.5% zero-shot accuracy, improving to 98.4% after fine-tuning with only 20% of target-domain data. The central contribution of this study is the combination of these three results within a single, transparently reported pipeline, directly addressing gaps left open by Saluja et al. [1], Al-Eryani et al. [2], Sakr et al. [6], and Cantone et al. [7], and demonstrating that lightweight, interpretable tree-based classifiers, deployed with per-class monitoring and periodic lightweight fine-tuning, represent a practically viable approach to DDoS detection despite the limitations disclosed in Section VI.

REFERENCES

[1] K. Saluja *et al.*, "Exploring Robust DDoS Detection: A Machine Learning Analysis with the CIC-DDoS2019 Dataset," in *Proc. IEEE 5th India Council International Subsections Conference (INDISCON)*, 2024, pp. 1–6.

[2] A. M. Al-Eryani, E. Hossny, and F. A. Omara, "Efficient Machine Learning Algorithms for DDoS Attack Detection," in *Proc. 6th International Conference on Computing and Informatics (ICCI)*, 2024, pp. 174–181.

[3] D. Kapil *et al.*, "Evaluating Machine Learning Approaches for DDoS Attack Detection Using CIC-DDoS2019," in *Proc. 2nd International Conference on Advanced Computing and Communication Technologies*, 2024, pp. 1–6.

[4] M. Meng, J. Zhou, J. Wang, and H. Yu, "Effective Intrusion Detection in Imbalanced Network Traffic Using XGBoost and LightGBM," in *Proc. IEEE International Conference on Artificial Intelligence and Extended and Virtual Reality (AIxVR)*, 2024, pp. 1–6.

[5] A. Abualhassan, I. Rashid, F. Binbeshr, and M. Imam, "DDoS Attack Detection in IoT: A Comparative Resource and Performance Analysis of Deep Learning and Machine Learning Models," *IEEE Access*, vol. 13, 2025.

[6] N. H. Sakr, A. Hikal *et al.*, "A Hybrid Approach for Efficient Feature Selection in Anomaly Intrusion Detection for IoT Networks," *The Journal of Supercomputing*, vol. 80, pp. 26942–26984, 2024.

[7] M. Cantone, C. Marrocco, and A. Bria, "Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study," *IEEE Access*, vol. 12, pp. 144489–144510, 2024.

[8] A. Prasad and S. Chandra, "Adaptive Sliding Window and LightGBM-Based DDoS Attack Detection Framework for IoT Networks," *Peer-to-Peer Networking and Applications*, Springer, 2025.

[9] R. S. Devi, R. Bharathi, and P. K. Kumar, "Investigation on Efficient Machine Learning Algorithm for DDoS Attack Detection," in *Proc. International Conference on Computer, Electrical and Communication Engineering*, 2023, pp. 1–5.

[10] T. F. Deressu, A. M. Beyene, and A. Y. Gemechu, "A Survey of Application Layer Distributed Denial of Service Detection: Approaches, Models, and Datasets," *Artificial Intelligence Review*, vol. 59, no. 6, 2026.

[11] M. B. Anley, A. Genovese, D. Agostinello, and V. Piuri, "Robust DDoS Attack Detection with Adaptive Transfer Learning," *Computers & Security*, vol. 145, 2024.

[12] B. Bouyeddou *et al.*, "A Systematic Literature Review on the Accuracy of Machine Learning-Based DDoS Detection Techniques in Software Defined Networking," *Information Retrieval Journal*, Springer, 2026.